Tecsup TECNOLOGÍA CON SENTIDO

ESTRUCTURA DE DATOS Y ALGORITMOS

Primer avance del proyecto final

Huellitas

Pernel Sebastian Rosas Quispe

Tecsup

Tercer ciclo · Sección C

Arequipa, 2026

Huellitas: primer avance del proyecto final

Contexto

Huellitas es un proyecto en Python con un mapa simulado, donde puedo agregar puntos para representar mascotas perdidas, avistamientos y refugios.

La idea nace de la necesidad de ordenar la información de una mascota perdida y los lugares donde pudo haber sido vista, por eso el mapa reúne esos registros junto con los refugios que pueden servir de apoyo.

Objetivo

Desarrollar una aplicación con interfaz gráfica que utilice un QuadTree para organizar los registros de mascotas perdidas en un mapa y permita buscarlos por zona.

¿Qué es un QuadTree?

Es una estructura de árbol que divide una región en cuatro partes y repite el proceso cuando una de esas partes necesita dividirse otra vez, siguiendo la idea de recursión que vimos en el curso.

En Huellitas, cada región normalmente guarda hasta cuatro puntos y, al agregar uno más, se divide en cuatro cuadrantes para repartirlos.

Relación con el curso

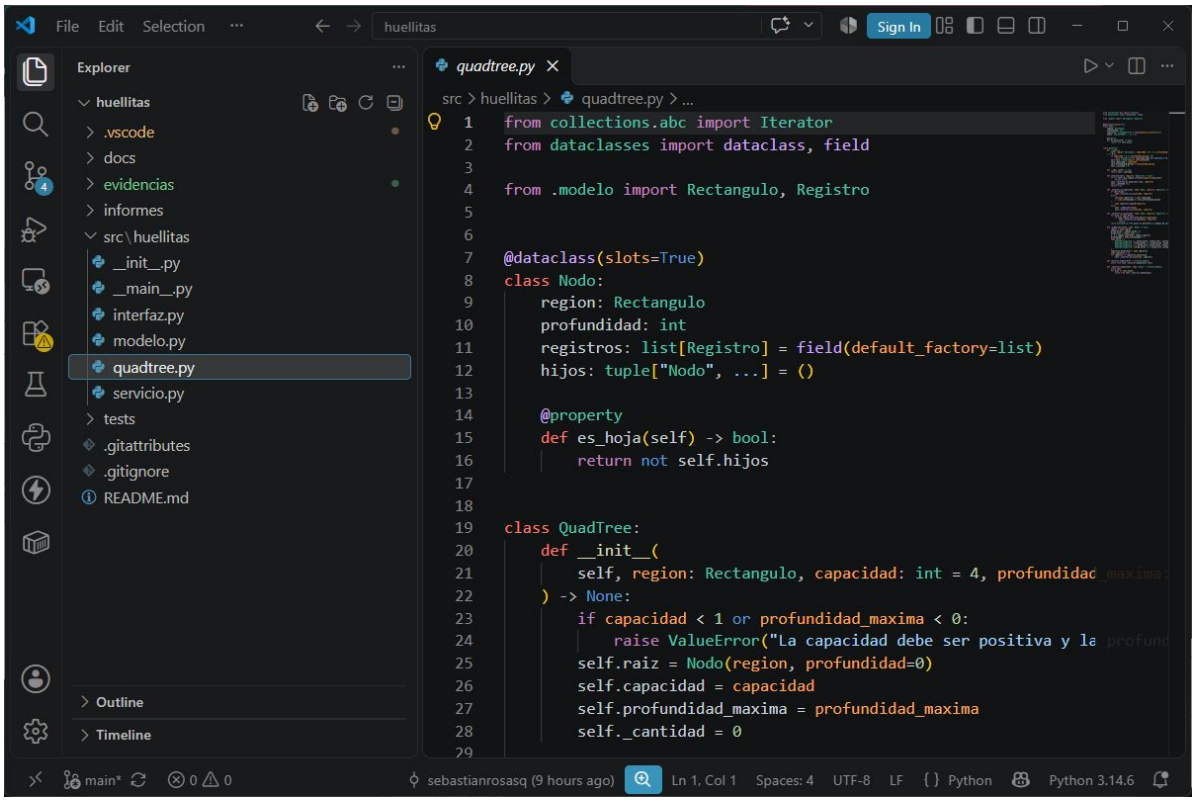
En este avance uso árboles, recursión y listas para guardar y repartir los registros, más adelante evaluaré una pila para deshacer el último registro y una cola para recorrer el árbol por niveles.

Organización del proyecto

El proyecto está abierto en Visual Studio Code, con carpetas para el código, las pruebas, el informe y las capturas.

Figura 1

Carpetas y archivos en Visual Studio Code



Nota: La carpeta src/huellitas reúne el código de la aplicación.

Cómo está separado el código

modelo.py contiene los datos de cada punto y registro, quadtree.py maneja los nodos y las divisiones, servicio.py conecta los registros con el árbol e interfaz.py dibuja la ventana con Tkinter.

Esta separación me permite revisar el QuadTree desde las pruebas sin tener que abrir la interfaz, mientras que __main__.py sirve para iniciar el programa.

Avance de la implementación

El QuadTree está escrito en Python y la ventana está hecha con Tkinter, mientras que Visual Studio Code permite trabajar con el código y ejecutar el programa.

Figura 2

Inserción de registros en el QuadTree

huellitas - quadtree.py

```python
def insertar(self, registro: Registro) -> bool:
    if not self.raiz.region.contiene(registro.posicion):
        return False
    self._insertar_en_nodo(self.raiz, registro)
    self._cantidad += 1
    return True

def _insertar_en_nodo(self, nodo: Nodo, registro: Registro) -> None:
    if nodo.hijos:
        self._insertar_en_hijo(nodo, registro)
    elif (
        len(nodo.registros) < self.capacidad
        or nodo.profundidad >= self.profundidad_maxima
    ):
        nodo.registros.append(registro)
    else:
        self._subdividir(nodo)
        self._insertar_en_hijo(nodo, registro)
```

Nota: Fragmento de quadtree.py exportado desde Visual Studio Code con CodeSnap.

Primero reviso que el punto esté dentro del mapa, luego el árbol busca la región donde corresponde guardarlo y aumenta la cantidad de registros.

Si el nodo ya tiene hijos, la inserción continúa en uno de ellos, si todavía tiene espacio guarda el registro y, cuando se llena, divide la región antes de agregar el nuevo punto.

También hay un límite de profundidad para que varios puntos en la misma posición no provoquen divisiones sin fin.

Subdivisión del mapa

Cuando una región se llena, el programa crea cuatro cuadrantes y reparte los puntos que ya estaban guardados en ella.

Figura 3

Creación de los cuatro cuadrantes

huellitas - quadtree.py

```
59  def _subdividir(self, nodo: Nodo) -> None:
60      region = nodo.region
61      mitad_ancho = region.ancho / 2
62      mitad_alto = region.alto / 2
63      x, y = region.izquierda, region.superior
64      profundidad = nodo.profundidad + 1
65      nodo.hijos = (
66          Nodo(Rectangulo(x, y, mitad_ancho, mitad_alto), profundidad),
67          Nodo(Rectangulo(x + mitad_ancho, y, mitad_ancho, mitad_alto), profundidad),
68          Nodo(Rectangulo(x, y + mitad_alto, mitad_ancho, mitad_alto), profundidad),
69          Nodo(Rectangulo(x + mitad_ancho, y + mitad_alto, mitad_ancho, mitad_alto), profundidad),
70      )
71      registros_anteriores = nodo.registros
72      nodo.registros = []
73      for registro in registros_anteriores:
74          self._insertar_en_hijo(nodo, registro)
75
```

Nota: Fragmento de quadtree.py exportado con CodeSnap, los registros pasan a sus nuevos cuadrantes.

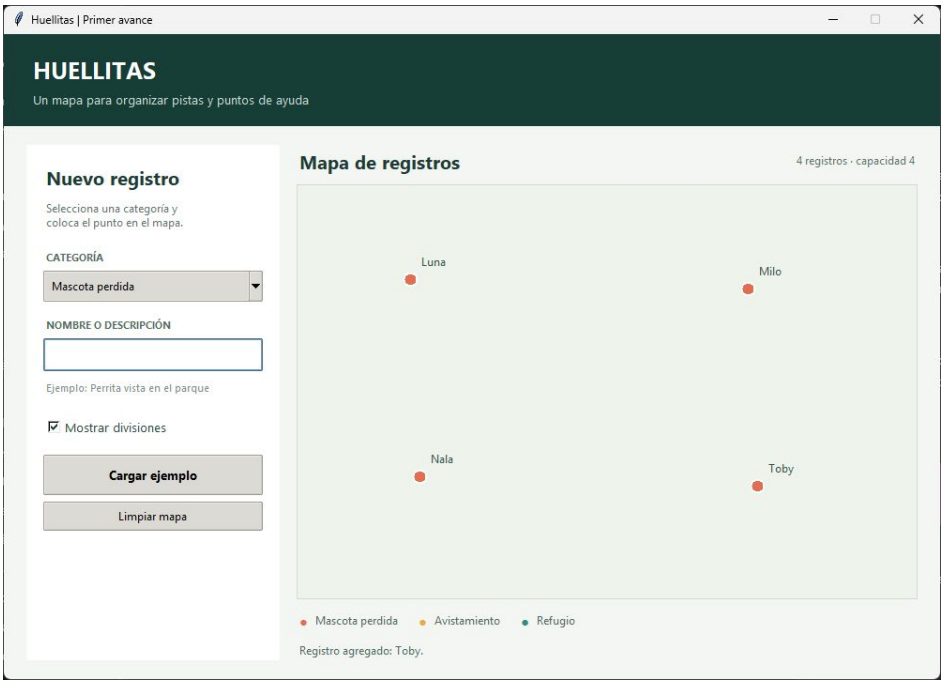
La división usa la mitad del ancho y del alto para formar las cuatro regiones, después crea sus nodos y vuelve a insertar los registros que tenía el nodo original.

Cada cuadrante incluye sus límites iniciales y deja fuera los finales, de esa manera un punto sobre una línea divisoria pertenece a una sola región.

Prueba de la división en la interfaz

Figura 4

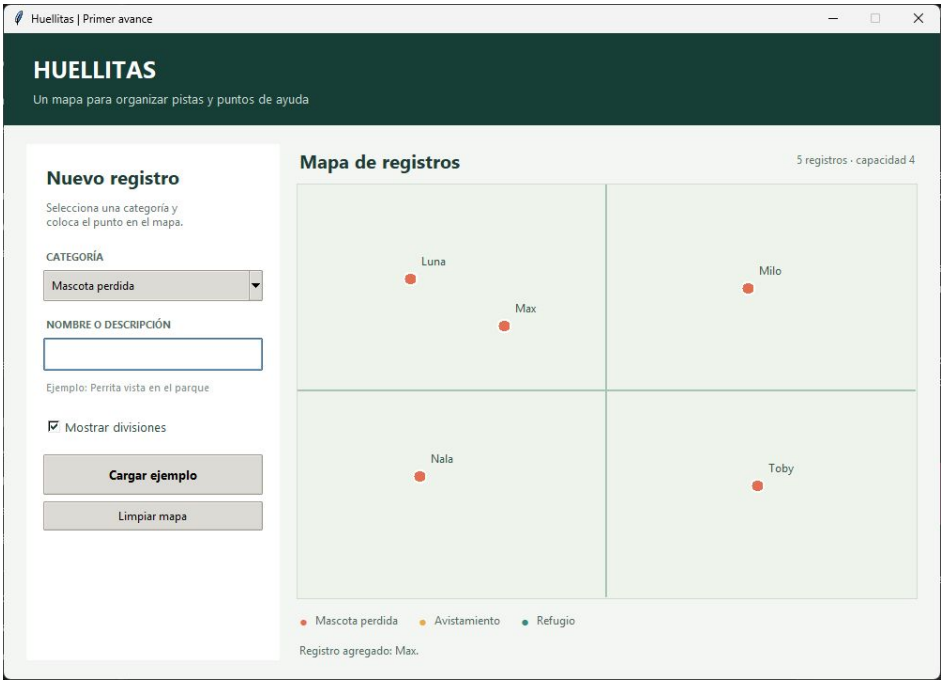
Mapa con cuatro registros



Nota: Con cuatro puntos, el mapa conserva una sola región.

Figura 5

Mapa después de agregar el quinto registro



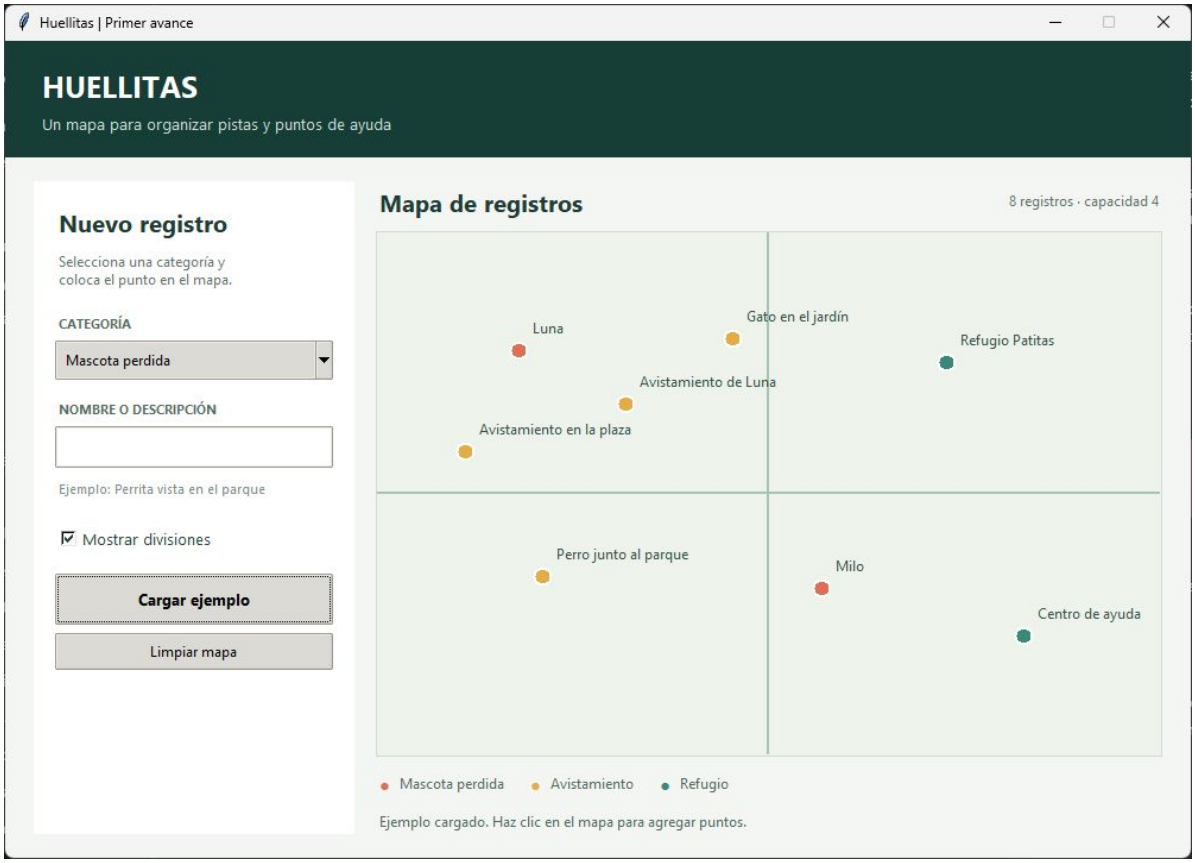
Nota: Al agregar a Max aparecen cuatro cuadrantes y permanecen los cinco registros.

Interfaz y datos de ejemplo

La ventana permite elegir una categoría, escribir una descripción y colocar el registro en el mapa, también tiene opciones para mostrar las divisiones, cargar un ejemplo y limpiar el mapa.

Figura 6

Ejemplo con mascotas, avistamientos y refugios



Nota: El ejemplo carga ocho registros y distingue las tres categorías por color.

Pruebas en la ventana

Al agregar un registro, su nombre aparece en el mapa y el contador aumenta, con el quinto punto aparecen las divisiones de la región.

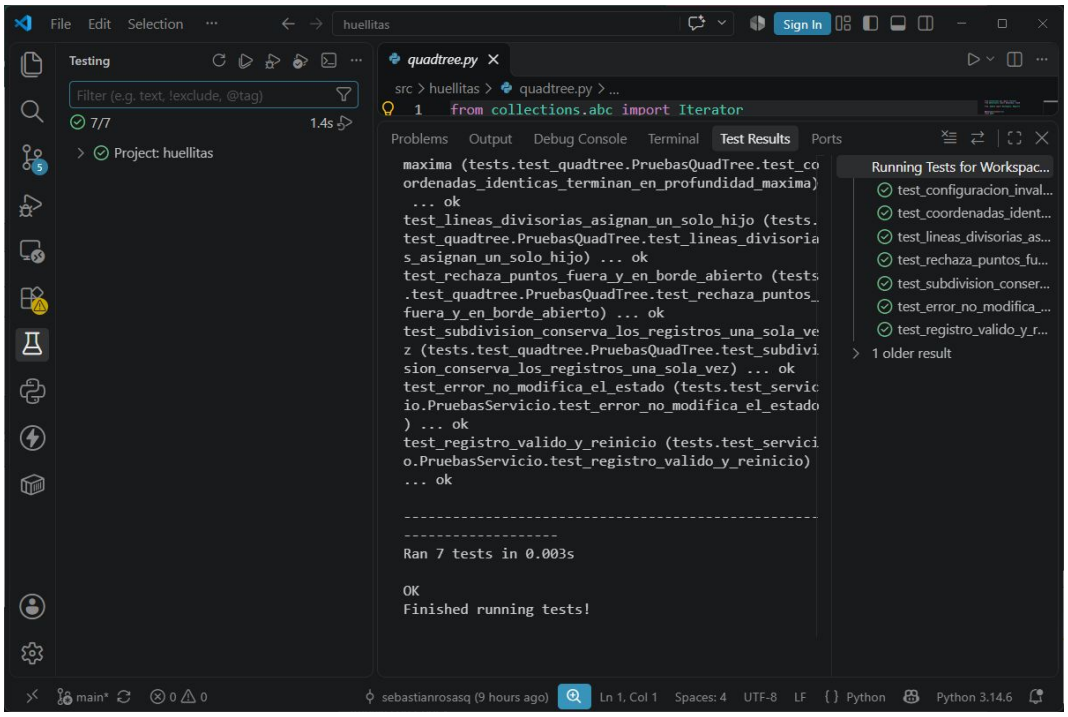
Al desactivar Mostrar divisiones, las líneas se ocultan y los puntos siguen en su lugar, al cargar el ejemplo aparecen ocho registros y Limpiar mapa deja el contador en cero.

Pruebas del código

Las siete pruebas ejecutadas desde Visual Studio Code terminaron correctamente, como aparece en la figura 7.

Figura 7

Resultado de las pruebas en Visual Studio Code



Nota: El panel muestra 7/7 y la salida de la ejecución termina con OK.

Las pruebas revisan la conservación de los registros, los límites del mapa y sus divisiones, también incluyen posiciones repetidas, datos inválidos y el reinicio.

Lo que falta

En la siguiente etapa quiero buscar los registros dentro de una zona seleccionada, después completaré las pruebas, el informe y el video para explicar el QuadTree y mostrar el programa.

Repositorio del proyecto

Usaré GitHub para guardar el código, las capturas y el informe, el repositorio público está en github.com/sebastianrosasq/proyecto-huellitas.